

Introducción a Python

Objetivo

Entender los conceptos básicos de **Python** y de **aritmética modular** necesarios para trabajar un **criptosistema RSA**.

¿Qué vamos a aprender?

Contenidos:

- Matemáticas mínimas: potencias, módulo e inverso modular.
- Python: variables y operadores básicos, operadores booleanos, funciones, listas y bucles.

Conceptos matemáticos básicos

Potencias

En Python, las potencias se escriben de la forma **a**b**, que representa a^b . Por ejemplo, 2^5 lo escribimos de la forma siguiente:

```
2 ** 5, (-7) ** 5
```

Operación módulo

La operación módulo devuelve el **resto** de dividir a entre b . Por ejemplo, $17 \bmod 5$ en Python se escribe **17%5**:

```
17 % 5    # 2
```

Inverso modular

x es inverso de a mod n si

$$a \cdot x \equiv 1 \pmod{n}$$

(existe si $\gcd(a, n) = 1$).

Python básico

Variables y tipos

```
edad = 13
nombre = "Ana"
print("Hola", nombre)
```

Operadores útiles

```
2 + 3
6 * 5
2 ** 10      # potencia
17 % 5       # módulo
```

Convertir letras números

```
ord("A"), chr(65)
```

Funciones

En Python, definimos funciones con `def nombre(...):`.

Ejemplo sin parámetro

```
def f():
    return "Hola, mundo."
f() # hay que llamar a la función
```

Ejemplo con parámetro

```
def f(x):  
    return x + 7  
print(f(10))
```

Como ves, si queremos devolver un valor usamos `return`.

Herramientas útiles en Python: `print`, booleanos y bucles

`print`: mostrar resultados

En Python usamos `print()` para **ver** lo que está pasando.

```
print("Hola")  
print("n =", 391)  
print("¿Funciona?", True)
```

Trucos útiles:

- Puedes imprimir varias cosas separadas por comas: `print("n =", 391, "e =", 3)`.
- Puedes personalizar el separador (`sep`) y el final (`end`):

```
print("A", "B", "C", sep=" - ")    # A - B - C  
print("Sin salto...", end="")      # no baja de línea  
print(" ahora sí")
```

Variables booleanas: `True` / `False`

Una variable booleana solo puede valer:

- `True` (verdadero)
- `False` (falso)

Ojo: en Python se escriben **con mayúscula**: `True` y `False`.

¿Cómo se obtienen?

Normalmente salen de **comparaciones**:

```
a = 10
b = 7
print(a > b)      # True
print(a == b)     # False
print(a != b)     # True
```

Operadores lógicos

Sirven para “juntar” condiciones:

```
x = 12
print(x > 0 and x < 20)  # True si cumple ambas
print(x < 0 or x == 12)  # True si cumple una
print(not (x == 12))     # False (porque sí es 12)
```

Listas en Python

¿Qué es una lista?

Una **lista** es una colección ordenada de elementos.

- Puede guardar números, texto, o incluso otras listas.
- El orden importa.
- Se escribe con corchetes: [...].

```
numeros = [10, 20, 30]
letras = ["A", "B", "C"]
mezcla = [1, "hola", True]

print(numeros)
print(letras)
print(mezcla)
```

Índices: cómo acceder a un elemento

En Python, las posiciones empiezan en **0**.

```
numeros = [10, 20, 30]
print(numeros[0]) # 10
print(numeros[1]) # 20
print(numeros[2]) # 30

# Índices negativos (desde el final)
print(numeros[-1]) # 30
print(numeros[-2]) # 20
```

Trozos (slicing): coger una parte

Con `lista[inicio:fin]` obtienes un “trozo”.

- inicio incluido
- fin **NO** incluido

```
numeros = [10, 20, 30, 40, 50]
print(numeros[1:4]) # [20, 30, 40]
print(numeros[:3]) # [10, 20, 30]
print(numeros[2:]) # [30, 40, 50]
```

Tamaño de una lista: `len()`

`len(lista)` devuelve cuántos elementos tiene.

```
numeros = [10, 20, 30, 40]
print(len(numeros)) # 4
```

Añadir y modificar elementos

Añadir al final: `append()`

```
numeros = [10, 20, 30]
numeros.append(40)
print(numeros) # [10, 20, 30, 40]
```

Añadir varios: `extend()`

```
numeros.extend([50, 60])
print(numeros) # [10, 20, 30, 40, 50, 60]
```

Insertar en una posición: insert()

```
numeros.insert(1, 99)
print(numeros) # [10, 99, 20, 30, 40, 50, 60]
```

Modificar un elemento por índice:

```
numeros[0] = 111
print(numeros)
```

Borrar elementos (con cuidado)

Borrar por índice: pop()

```
numeros = [10, 20, 30]
x = numeros.pop(1)      # quita el de la posición 1
print(x)                # 20
print(numeros)          # [10, 30]
```

Borrar por valor: remove()

```
numeros = [10, 20, 30, 20]
numeros.remove(20)      # quita la primera aparición de 20
print(numeros)          # [10, 30, 20]
```

Vaciar la lista: clear()

```
numeros.clear()
print(numeros)          # []
```

Si intentas borrar algo que no existe, Python da error.

Bucles for y while

Bucle **for**: repetir un número de veces

Un **for** repite algo **un número de veces** o recorre una lista.
La estructura general:

```
for i in range(inicio, fin, paso):  
    ...
```

Repetir 5 veces:

```
for i in range(5):  
    print("Vuelta", i)
```

range(5) genera: 0, 1, 2, 3, 4.

Repetir de 1 a 5:

```
for i in range(1, 6):  
    print(i)
```

Recorrer una lista

```
valores = [3, 5, 7, 9]  
for v in valores:  
    print("Valor:", v)
```

Bucle **while**: repetir mientras se cumpla una condición

Un **while** repite **mientras** una condición sea **True**.
La estructura general:

```
while condicion:  
    ...
```

Ejemplo: contar hasta 5

```
contador = 1
while contador <= 5:
    print("contador =", contador)
    contador += 1
```

¿Por qué es importante cambiar algo dentro?

Porque si la condición nunca cambia... ¡bucle infinito!

```
# EJEMPLO DE LO QUE NO HAY QUE HACER (bucle infinito)
# x = 1
# while x < 10:
#     print(x)
```

Construcción de listas

A mano

Se escribe directamente con corchetes:

```
edades = [12, 13, 14]
colores = ["rojo", "verde", "azul"]
print(edades, colores)
```

También puedes repetir valores con *:

```
ceros = [0] * 5
print(ceros) # [0, 0, 0, 0, 0]
```

range()

`range()` genera una secuencia de enteros. Para convertirla en lista usamos `list(...)`.

```
lista1 = list(range(5))          # 0,1,2,3,4
lista2 = list(range(1, 6))       # 1,2,3,4,5
lista3 = list(range(0, 11, 2))   # 0,2,4,6,8,10
print(lista1)
print(lista2)
print(lista3)
```

Recuerda: `range(inicio, fin, paso)` y el fin no se incluye.

Con un bucle y `append()`

Construimos una lista poco a poco:

```
cuadrados = []
for i in range(1, 6):
    cuadrados.append(i**2)
print(cuadrados) # [1, 4, 9, 16, 25]
```

Esta forma es muy clara para empezar.

Listas por comprensión

Hace lo mismo que un `for` + `append`, pero en una sola línea.

```
cuadrados = [i**2 for i in range(1, 6)]
pares = [i for i in range(21) if i % 2 == 0]
print(cuadrados)
print(pares)
```

Este método se parece mucho a las matemáticas cuando se define una función con su dominio.

A partir de texto: `split()`

`split()` separa un texto en partes y devuelve una lista.

```
frase = "hola que tal estas"
palabras = frase.split() # separa por espacios
print(palabras)

csv = "3,5,7,9"
partes = csv.split(",")
print(partes) # ojo: son textos
```

Si quieres convertir a números:

```
csv = "3,5,7,9"
nums = [int(x) for x in csv.split(",")]
print(nums)
```

arange() (NumPy)

arange() pertenece a la librería **NumPy**. Se usa mucho en ciencia de datos.

- `np.arange(...)` devuelve un **array de NumPy**, no una lista.
- Si necesitas una lista, convierte con `list(...)`.

```
import numpy as np

arr = np.arange(0, 1.0, 0.2)    # 0.0, 0.2, 0.4, 0.6, 0.8
print(arr)

lista = list(arr)
print(lista)
```

Nota: la **primera vez** que se importa NumPy en el navegador puede tardar unos segundos en cargarse.

Retos

Comprueba lo que has aprendido

1. Crea una función `es_par` que sea `True` si `n` es par, y `False` si es impar. Pista: `n % 2 == 0`.

```
# Escribe aquí tu programa:
```

2. Haz un `for` que imprima los números del 1 al 10.

```
# Escribe aquí tu programa:
```

3. Haz un `while` que vaya sumando 1 a un contador hasta llegar a 10.

```
# Escribe aquí tu programa:
```

4. Dada una lista `[2, 4, 6, 8]`, crea otra con el doble de cada número recorriéndola con `for`.

```
# Escribe aquí tu programa:
```

5. Cambia el último elemento de una lista por otro (usando [-1]).

```
# Escribe aquí tu programa:
```